R.M.K
GROUP OF ENGINEERING INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

R.M.K
GROUP OF
INSTITUTIONS

22IT958

AGENTIC AI

Department: IT

Batch/Year: 2023-27 / III YEAR / V SEMESTER

Prepared / Updated by:

Dr K Saravanan/Prof/IT/RMKEC
Ms Divya/AP/IT/RMKEC

Date: 20.01.2026

R.M.K GROUP OF INSTITUTIONS

Table of Contents

- Course Objectives
- Pre Requisites
- Syllabus
- Course outcomes
- CO- PO/PSO Mapping
- Lecture Plan
- Activity based learning
- Unit 1 :
- Lecture Notes
- Lecture Notes – Links to Videos
- Lecture Notes – e book reference
- Lecture Notes – PPTs
- Lecture Notes - Quiz
- Lecture Notes - References
- Assignments
- Part A Q & A (with K level and CO)
- Part B Qs (with K level and CO)
- Supportive online Certification courses
- Real time Applications in day to day life and to Industry
- Contents beyond the Syllabus
- Assessment Schedule
- Prescribed Text Books & Reference Books
- Gate question and answers
- Mini Project suggestions

R.M.K GROUP OF INSTITUTIONS

Course Objectives

- To introduce the basics of Agentic AI, including autonomous agents, multi-agent systems, and ethics.
- To help students learn LLM capabilities and use prompting methods like ReACT, CoT, and Plan-Execute.
- To enable students to design AI systems using LangChain, LangGraph, and Agentic AI patterns.
- To give hands-on practice in building real AI applications such as chatbots, multimodal systems, and Agentic RAG solutions.
- To make students familiar with low-code and no-code AI platforms for quick agent development and deployment.

R.M.K GROUP OF INSTITUTIONS

Pre Requisites

- **22CS101 – Generative AI**
- **22EC101 – Introduction to AI**

R.M.K
GROUP OF
INSTITUTIONS

Syllabus

UNIT I INTRODUCTION TO AGENTIC AI 6+6

Agentic AI Introduction - AI Agents vs. Agentic AI -Comparison: Agentic AI, Generative AI, and Traditional AI -Agentic AI Building Blocks - Autonomous Agents, Human in the Loops Systems

Single and Multi Agent AI Systems - Agentic AI Frameworks Overview, Ethical and Responsible AI, Agentic AI Best Practices

AI Implementation Success Stories: Text generation, Translation, Summarization, Question answering, and Code generation, Reasoning, Sentiment Analysis, Classification, Translation, Image and Audio capabilities

LLM Introduction – types of LLM – LLM Capabilities - Prompt Engineering – Introduction – Elements of Prompting Engineering – Different types of Prompts – ReACT, CoT, Self Ask, Plan & Execute.

List of Exercise:

1. Develop a Python script for image information extraction, image classification, and image segmentation

R.M.K GROUP OF INSTITUTIONS

Syllabus

UNIT II AGENTIC AI FRAMEWORK MODULES & DESIGN PATTERNS 6+6

Agentic AI System Modules- Perception Module, Cognitive Module, Action Module, Learning Module, Collaboration Module, Security Module, Agentic AI Design Patterns-Reflection Pattern, Tool Use Pattern, Planning Pattern, ReAct (Reasoning and Acting) and ReWOO (Reasoning with Open Ontology),Multi Agent Pattern, Design Considerations.

List of Excersise:

1. Use any LLM (ChatGPT / Gemini / Claude) to answer the same question with different prompting styles and compare responses.

2. Create a Python script (or notebook) that uses an LLM to do:

- Text summarization
- Sentiment analysis
- Question answering
- Students must log inputs, outputs, and observe how prompts affect results.

Syllabus

UNIT III LANGCHAIN & LANGGRAPH FOR AGENT DEVELOPMENT 6+6

Langchain – Prompts, Chain, Tools and Agents, Memory, Runnables, LCEL, Integration with LLM Service Providers (OpenAI, Google Gemini, Ollama, Hugging Face, Claude). LangGraph – Introduction to LangGraph, State and Memory, State Schema, State Reducer, Multiple Schemas, Trim and Filter Messages, Memory and External Memory. UX and Human-in-the-Loop (HITL), Building Agent with LangGraph, Long Term Memory, Short vs. Long Term Memory, Memory Schema, Deployment.

List of Exercise:

Generate Text Using an LLM

Create a Simple Prompt Template

Build a LangChain agent that:

(i) Reads a table schema

(ii) Converts natural language to SQL

Create a Simple LangGraph Workflow

Create a Chatbot With Memory

Syllabus

UNIT IV	MULTI-AGENT FRAMEWORKS & AGENTIC RAG	6+6

Embeddings & embedding models Cosine similarity, dot product, Euclidean distance High-dimensional search fundamentals Dense vs sparse vectors Indexing basics (HNSW, IVF, Flat index) HNSW internal workings Approximate Nearest Neighbour (ANN).

What is Agentic RAG?, Agentic RAG vs. Traditional RAG,Agentic RAG Architecture and Components,Variants of Agentic RAG,Applications of Agentic RAG,Agentic RAG with Llamaindex, Agentic RAG with Cohere

List of Exercise:

Create a Personalized Study Material Generator using Agentic RAG

Generate a Student Attendance & Academic Progress Query System using Agentic RAG

UNIT V	AGENTIC AI FRAMEWORKS	6+6

CrewAI – Introduction, Architecture, Task Orchestration, Agent Communication, Integration, Prompt Engineering in CrewAI, Security and Trust - AutoGen – Introduction, Core, Agent Design and Behaviour, Agent Communication,Task Executor, Workflows

Model Context Protocol (MCP). Introduction to LC/NC Platforms - n8n - langflow

List of Exercise:.

Create two simple agents (Writer + Reviewer) and a Crew that generates and improves a paragraph.

Build a Crew that answers questions from a PDF (syllabus, notes, textbook chapter).

Create a Crew that: (i) Searches online for a topic. (ii) Summarizes the result into 5 bullet points

Course outcomes

Course Outcome Statements	Blooms Level
Explain the basics of Agentic AI and how it differs from Traditional and Generative AI	K2
Apply simple prompt engineering methods like ReAct (Reason + Act), CoT, Self-Ask, and Plan-Execute to improve AI agent performance.	K4
Design AI workflows using LangChain modules such as prompts, chains, memory, tools, and agents	K4
Build AI agents using LangGraph with concepts like state, memory, reducers, and Human-in-the-Loop steps.	K4
Configure different TCP/IP services	K5
Create multi-agent and Agentic RAG applications using tools like LlamaIndex, Cohere, CrewAI, and AutoGen.	K4

INSTITUTIONS

CO- PO/PSO Mapping

Course Outcomes (COs)		Programme Outcomes (POs), Programme Specific Outcomes (PSOs)														
		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
		K3	K4	K5	K5	K3/K5	A2	A3	A3	A3	A3	A3	A2	K3	K3	K3
C212.1	K2	2	1											3	2	2
C212.2	K4	3	3	2	2									3	2	2
C212.3	K3	3	2	1	1									2	3	2
C212.4	K4	3	3	2	2									2	2	3
C212.5	K3	3	2	1	1									3	3	3
C212		2.8	2.2	1.2	1.2									2.6	2.4	2.4

GROUP OF INSTITUTIONS

UNIT III
LANGCHAIN & LANGGRAPH FOR AGENT DEVELOPMENT

R.M.K
GROUP OF
INSTITUTIONS

Lecture Plan

UNIT-II Transmission Media and Switching					
Sl.No.	**Topics**	**No. of Periods**	**Pertaining CO**	**Taxonomy Level**	**Mode of Delivery**
1	Langchain – Prompts	1	CO2	K3	ICT
2	LCEL, Integration with LLM Service Providers	1		K3	
3	LCEL, Integration with LLM Service Providers	1		K3	
4	LangGraph – Introduction to LangGraph	1		K3	
5	State and Memory	1		K3	
6	State Schema	1		K4	
7	State Reducer	1		K3	
8	Multiple Schemas	1		K3	
9	Trim and Filter Messages	1		K3	
10	Memory and External Memory	1		K4	
11	Building Agent with LangGraph	1		K3	
12	Memory Schema, Deployment.	1		K3	

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

UNIT 3: LANGCHAIN & LANGGRAPH FOR AGENT DEVELOPMENT

LangChain – Introduction

LangChain is a **framework used to build applications powered by Large Language Models (LLMs)** such as OpenAI GPT, Google Gemini, Claude, and others. It provides structured components that help developers **connect prompts, models, tools, memory, and logic** in an organized way. Instead of writing complex LLM logic from scratch, LangChain offers reusable building blocks that make AI application development easier, scalable, and maintainable.

LangChain is widely used for **chatbots, question-answering systems, agents, document analysis, and workflow automation**.

Components of LangChain

- **Chains:** Chains define sequences of actions, where each step can involve querying an LLM, manipulating data or interacting with external tools. There are two types:
  - **Simple Chains**: A single LLM invocation.
  - **Multi-step Chains**: Multiple LLMs or actions combined, where each step can take the output from the previous step.

R.M.K GROUP OF INSTITUTIONS

- **Prompt Management:** LangChain facilitates managing and customizing prompts passed to the LLM. Developers can use **PromptTemplates** to define how inputs and outputs are formatted before being passed to the model. It also simplifies tasks like handling dynamic variables and prompt engineering, making it easier to control the LLM's behavior.
- **Agents:** Agents are autonomous systems within LangChain that take actions based on input data. They can call external APIs or query databases dynamically, making decisions based on the situation. These agents leverage LLMs for decision-making, allowing them to respond intelligently to changing input.
- **Vector Database:** LangChain integrates with a vector database which is used to store and search high-dimensional vector representations of data. This is important for performing similarity searches, where the LLM converts a query into a vector and compares it against the vectors in the database to retrieve relevant information.
- **Models:** LangChain is model-agnostic meaning it can integrate with different LLMs such as OpenAI's GPT, Hugging Face models, DeepSeek R1 and more. This flexibility allows developers to choose the best model for their use case while benefiting from LangChain's architecture.
- **Memory Management:** LangChain supports memory management allowing the LLM to "remember" context from previous interactions. This is especially useful for creating conversational agents that need context across multiple inputs.

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Prompts in LangChain

A **prompt** is the input instruction given to a language model that guides how it should respond. In LangChain, prompts are treated as **first-class objects**, meaning they can be created, reused, parameterized, and combined easily.

LangChain allows developers to create **prompt templates**, where placeholders are used instead of fixed values. This makes prompts flexible and reusable.

- Prompts define **what the model should do**
- Can include **instructions, examples, and context**
- Support **variables (dynamic inputs)**
- Help maintain **consistent outputs**

Example: A prompt template for summarization:

"Summarize the following text in simple terms: {text}"

When {text} is replaced with actual content, the model generates a summary.

Chains in LangChain

A **Chain** is a sequence of steps where the **output of one component becomes the input of the next**. Chains help structure complex workflows involving prompts, LLMs, and tools.

Instead of calling an LLM once, chains allow **multiple operations in a defined order**.

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Key Points

- Chains connect **prompts** → **models** → **outputs**
- Can contain **single or multiple steps**
- Improve **readability and modularity**
- Useful for multi-step reasoning

Example

User asks a question

Prompt formats the question

LLM generates an answer

Output is refined or summarized

This sequence forms a **chain**.

Tools and Agents in LangChain

Tools

Tools are **external functions or APIs** that an LLM can use to perform specific tasks, such as searching the web, performing calculations, or accessing databases.

Agents:

An **Agent** is an intelligent controller that decides:

- Which **tool to use**
- In what **order**
- Based on the **user's query**

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Agents use **reasoning and decision-making** to solve complex tasks dynamically.

- Tools extend LLM capabilities
- Agents choose tools automatically
- Suitable for **autonomous AI systems**
- Enable real-world interactions

Example - User: *"What is the weather in Chennai today?"*

Agent decides to:

1.Use a **weather API tool**

2.Fetch real-time data

3.Return the answer

Memory in LangChain

Memory allows LangChain applications to **store and recall past interactions**. Without memory, an LLM treats each request as independent.

LangChain supports **short-term memory** (conversation history) and **long-term memory** (stored data across sessions).

•Enables **context-aware conversations**

•Improves personalization

•Can store summaries or full chat history

•Essential for chatbots and assistants

Example

User: *"My city is NewYork "*

Later: *"What is my city?"*

With memory enabled, the model answers correctly.

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Runnables in LangChain

Runnables are **standardized executable components** in LangChain that can be composed and executed in a consistent way. They allow developers to treat prompts, models, chains, and tools as **callable objects** that can be combined easily.

Key Points

•Unified execution interface

•Enable easy composition

•Support parallel and sequential execution

•Improve flexibility

Example

A prompt runnable + LLM runnable → combined and executed as one runnable pipeline.

How LangChain Works?

1. User Query

The process begins when a user submits a query or request.For example, a user might ask, "What's the weather like today?" This query serves as the input to the LangChain pipeline.

2. Vector Representation and Similarity Search

Once the query is received, LangChain converts it into a vector representation using embeddings. This vector captures the semantic meaning of the query.The vector is then used to perform a similarity search in a vector database. The goal is to find the most relevant information or context stored in the database that matches the user's query.

R.M.K GROUP OF INSTITUTIONS

3. Fetching Relevant Information

Based on the similarity search, LangChain retrieves the most relevant data or context from the database. This step ensures that the language model has access to accurate and contextually appropriate information to generate a meaningful response.

4. Generating a Response

The retrieved information is passed to the language model (e.g., OpenAI's GPT, Anthropic's Claude or others). The LLM processes the input and generates a response or takes an action based on the provided data.

Integration with LLM:

- To elevate the capabilities of your software product, you can integrate an LLM into its foundation. However, this is a delicate process that requires careful planning and execution. Otherwise, you may risk breaking pre-existing features or bloating the software with a tool that barely works.
- To begin, you'll need to identify the tasks you want the LLM to perform in the software. This may range from answering user queries to automating user tasks. Once you've defined your objectives, fine-tuning the LLM using a relevant training process is required.
- Next, you'll need to integrate the LLM into your software, either by embedding it directly into its functionality or by connecting it externally via an API. Note that a lot of testing is needed to ensure the LLM performs as expected. This involves feeding the LLM with various inputs and assessing its outputs for accuracy and relevance.

Lecture Notes

How To Integrate LLMs:

Attempting LLM implementation can be an arduous process. The upside is that it's universal for all types of models and sizes businesses may want to add to their systems or software solutions. There are several components and integration methods to consider.

❁ Choose the proper LLM for your business case

The large language model is central to this process, and before integration, we strongly recommend choosing the best type of LLM that will fit your business needs. The discovery phase, business analysis, and analysis of LLMs' statistics are necessary for the proper integration.

It is crucial to find the proper open-source or closed-source LLM at the beginning and avoid complex fine-tuning processes in later stages. Springs' team has already experienced thiswhile building our first AI chatbot.

❁ Define the API Integration Method and get API access

The next necessary step in LLM integration is to decide whether to use the LLM provider's API or deploy the model locally.

Consider factors like scalability, maintenance overhead, and resource constraints when making this decision.

Lecture Notes

- **Prepare the needed data:**

Depending on your business use case, preprocess or format your input data before sending it to the LLM. Tasks may include tokenization, normalization, or filtering out irrelevant information.

- Setup interaction with your LLM and handle errors: The first thing you need to do here is to allocate AI developers to write code to interact with the LLM through the API and construct HTTP requests with appropriate headers, parameters, and payload.
- Test and deploy your LLM: The next important step is QA testing and LLM deployment. At this point, you need to create comprehensive test cases to validate your integration. This can be done by a professional QA team as well.
- **Monitor and support your product:** The last important part of any product development is **ongoing software support**, monitoring, and maintenance.

Top LLM Models

A **Large Language Model (LLM)** refers to a neural network equipped with **billions of parameters** and trained on **large-scale, unlabelled text data**.

The training process typically uses **self-supervised** or **semi-supervised learning techniques**, enabling the model to understand and generate human-like language.

R.M.K GROUP OF INSTITUTIONS

GPT-4

- As of 2024, OpenAI's GPT-4 stands out as one of the leading Large Language Models (LLMs) in the AI market. It was launched in March 2023.
- Although the exact parameter count of GPT-4 has not been publicly disclosed, there are rumors suggesting that it may contain more than 170 trillion parameters. Regardless of the exact size, GPT-4 has demonstrated exceptional capabilities across multiple domains.

GPT-4 excels in:

- Complex reasoning tasks
- Advanced coding and programming
- Various academic and professional domains
- Achieving near **human-level performance** in many skills
- A key milestone of GPT-4 is that it is the **first multimodal model**, capable of accepting both **text and image inputs**.
- Additionally, GPT-4 significantly addresses the issue of **hallucinations** and improves **factual accuracy**. In factual evaluations across multiple categories, GPT-4 outperforms **ChatGPT-3.5**, achieving scores close to **80% accuracy**.

R.M.K GROUP OF INSTITUTIONS

Features of GPT-4

- **Massive Scale:** GPT-4 is built on a very large-scale architecture, allowing it to handle complex language understanding and generation tasks efficiently.

- **Advanced Natural Language Understanding:** It exhibits enhanced capabilities in understanding complex language structures, nuances, and context, leading to more accurate and contextually appropriate responses.

- **Fine-Tuning Flexibility**: GPT-4 offers flexibility in fine-tuning for specific tasks or domains, making it adaptable to various NLP applications.

Gemini

Google's new AI, Gemini, seems to be stepping up the game against ChatGPT. Released in December 2023 it was built from the ground up to be multimodal, which means it can generalize and seamlessly understand, operate across and combine different types of information including text, code, audio, image and video. Gemini Ultra is the first model to outperform human experts on MMLU (massive multitask language understanding), which uses a combination of 57 subjects such as math, physics, history, law, medicine and ethics for testing both world knowledge and problem-solving abilities.

Developers and enterprise customers can access Gemini Pro via the Gemini API in Google AI Studio or Google Cloud Vertex AI.

Feature of Gemini

•Conversational AI Focus: Gemini emphasizes improving conversational AI by better understanding context and generating more human-like responses in dialogues.

•Contextual Sensitivity: It exhibits enhanced sensitivity to context shifts within conversations, leading to more coherent and contextually appropriate responses.

•Multimodal Integration: Gemini integrates multiple modalities such as text, images, and audio to enrich the conversational experience and generate more comprehensive

LLaMA

LLaMa, or Large Language Model Meta AI, emerges as a significant development in the realm of Large Language Models by Meta AI. The inaugural release of LLaMa in February 2023, with the largest version of 65 billion parameters in size. Since its unveiling, Meta's introduction of the LLaMa family of large language models (LLMs) has become a valuable asset for the open-source community. The diverse range of LLaMA models, spanning from 7 billion to 65 billion parameters, has demonstrated superior performance compared to other LLMs, including GPT-3, across various benchmarks.

Lecture Notes

- An undeniable advantage of LLaMA models lies in their open-source nature, empowering developers to easily fine-tune and create new models tailored to specific tasks. This approach fosters rapid innovation within the open-source community, leading to the continuous release of new and enhanced LLM models.

Feature of Llama

- Long-Term Language Understanding: LLaMA specializes in long-term language understanding and reasoning, enabling it to grasp complex relationships and concepts across extended passages of text.
- Reasoning Capabilities: It incorporates advanced reasoning capabilities, allowing it to infer implicit information, draw logical conclusions, and answer complex questions.

Hugging Face

- Hugging Face is one of the best platforms for machine learning, and artificial intelligence (AI) models. Using the Hugging Face API, we can easily interact with various pre-trained models for tasks like text generation, translation, sentiment analysis, etc.

Features of Hugging Face

- Ease of use: We don't have to spend hours building models from scratch. The API streamlines the process, making it accessible even to beginners.

R.M.K GROUP OF INSTITUTIONS

- Pre-trained Models: Hugging Face offers a treasure trove of pre-trained models, which means we can pick one that suits what we need and start using it right away.
- Time-Saving: We can integrate AI into our projects with the Hugging Face API and we can focus on what really matters that is developing our application rather than getting bogged down in the intricacies of machine learning.

Claude v1

- It's not as popular LLM as compared to GPT or Llama but Claude is a powerful LLM developed by Anthropic, former co-founded by OpenAI employees. It's a newbie on the Large Language Model block that's outperforming PaLM 2 in benchmark tests and offering a 100k token context window for the first time ever. It's in competition with GPT-4 and scored 7.94 in the MT-Bench test, while GPT-4 scored 8.99 and in the MMLU benchmark as well, Claude v1 secures 75.6 points, and GPT-4 scores 86.4. Previously said that is offers 100k tokens which is equivalent to about 75,000 words, this means that you can easily load a full-length book into Claude's context window, it would still understand it and create text in response to your prompts.

Features of Claude v1

- Understanding Complex Structures: Claude v1 excels in understanding complex language structures, including nuanced expressions, idiomatic phrases, and syntactic variations.

- Coherent Responses: It generates coherent and contextually relevant responses across diverse contexts, maintaining coherence and consistency in dialogues and interactions.
- Task Adaptability: Claude v1 is adaptable to various NLP tasks and domains, offering flexibility in application and integration into different

LCEL (LangChain Expression Language)

- LCEL stands for LangChain Expression Language. It is a declarative and expressive syntax provided by LangChain to build and connect different components such as prompts, language models, tools, and output parsers in a clean and readable manner. LCEL allows developers to define workflows by describing what should happen, rather than writing long procedural code. LCEL uses a pipeline-style approach, where each component is connected to the next, forming a clear flow of execution. This makes AI application development more modular, maintainable, and scalable.

- Declarative way to define chains
- Easy to read and write
- Supports sequential and parallel execution
- Components are reusable
- Reduces boilerplate code

R.M.K GROUP OF INSTITUTIONS

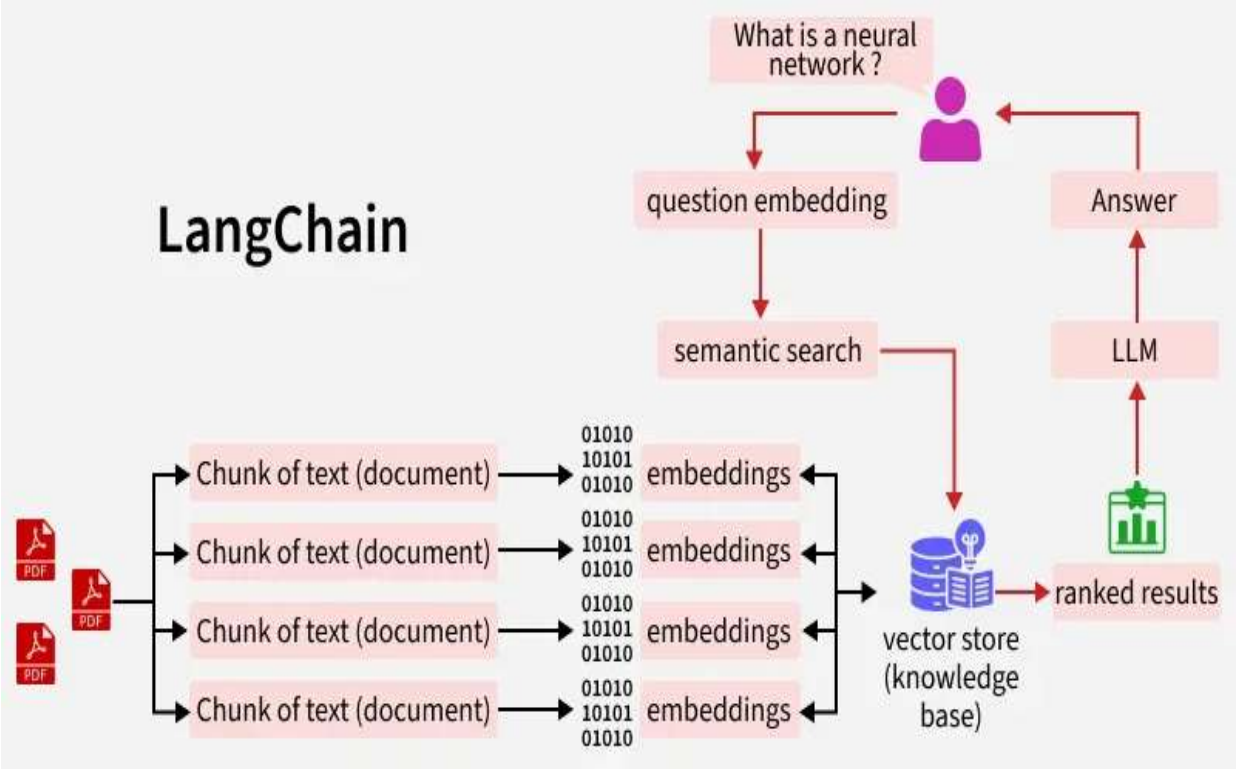
Advantages of LCEL

- Improves code clarity
- Easy debugging
- Supports advanced workflows
- Suitable for large applications

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

LangChain provides a **powerful, modular, and flexible framework** for building intelligent AI applications. By combining **prompts, chains, tools, agents, memory, runnables, and LCEL**, developers can create scalable and intelligent systems that integrate seamlessly with various LLM providers. LangChain simplifies complex workflows and enables the development of advanced AI solutions in a structured and maintainable manner.



Lecture Notes

LangGraph

LangGraph is a framework built on top of LangChain that is used to design **stateful, multi-step, and cyclic AI workflows**. Unlike simple chains that execute in a straight line, LangGraph allows workflows to be represented as **graphs**, where each node represents a computation and edges represent the flow of control. This makes LangGraph suitable for **complex agent systems, decision-making processes, and long-running interactions**.

LangGraph is especially useful for building **agents with memory, branching logic, and human-in-the-loop (HITL) systems**.

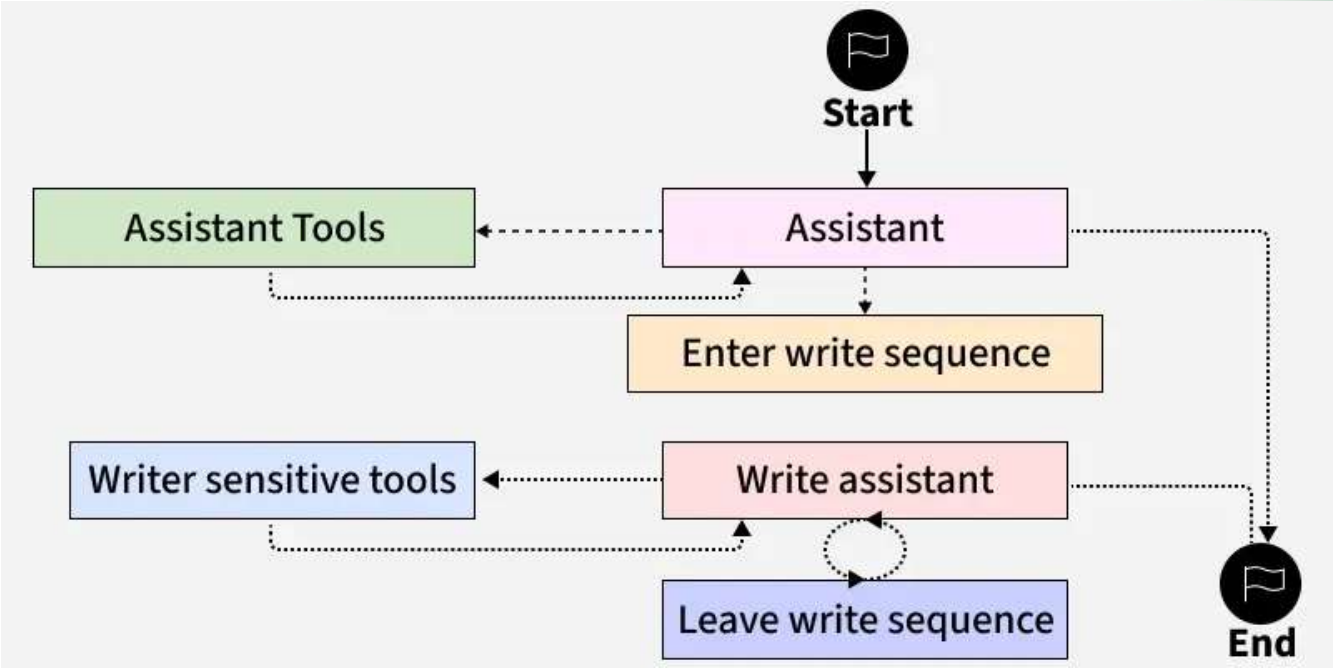
1. Introduction to LangGraph

LangGraph enables developers to define workflows as **directed graphs** rather than linear sequences. Each node performs a task such as calling an LLM, updating memory, or invoking a tool. The graph can loop, branch, or pause based on the current state.

- Graph-based execution model
- Supports loops and conditional paths
- Ideal for complex agent behavior
- Works seamlessly with LangChain components

R.M.K GROUP OF INSTITUTIONS

Lecture Notes



Workflow of LangGraph: The diagram below shows how LangGraph structures its agent-based workflow using distinct tools and stages.

Here's a step-by-step interpretation of the flow:

- **Start:** The process begins with the agent (Assistant) initiating an interaction or task.
- **Assistant:** This is the central node managing the overall workflow. It controls the movement between tools and sequences based on the current state.
- **Enter Write Sequence:** If the task requires writing assistance like generating content, the workflow enters a dedicated writing sequence.
- **Write Assistant**: This specialized module focuses on the writing process. It may loop with tools for refining or editing before completing the sequence.

R.M.K GROUP OF INSTITUTIONS

- **Leave Write Sequence:** Once the writing task is completed, the system exits the write mode and returns to the normal operational flow.
- **Writer-Sensitive Tools & Assistant Tools:** These nodes provide specialized capabilities to support the process. Based on the current system state, the Assistant routes tasks to appropriate tools that either enhance writing quality or perform sensitive operations.
- **End:** The process concludes after the desired outcome is achieved and all required tools have been successfully executed.

Components of LangGraph

- These core components work together smoothly to help developers build, customize and manage complex AI-driven workflows.
- **Monitoring mechanism:** Human-in-the-loop (HITL) ensures humans remain part of the decision-making process. It improves machine learning accuracy by using critical data points instead of relying on random sampling.
- **Stateful graphs:** Each node represents a step in computation and carries forward information from previous steps. This enables continuous, contextual processing of data throughout the workflow.

Lecture Notes

- **Cyclical graphs:** Graphs that contain loops is used for workflows where certain steps may repeat. It becomes important for complex agent run-times.
- **Nodes:** The individual components or agents within a workflow is called node. They act like "actors" performing tasks or calling tools (e.g., a ToolNode for tool integration).
- **Edges:** Edges determine which node should run next. They can follow fixed paths or branching conditions based on the system state.
- **RAG (Retrieval-Augmented Generation):** RAG enhances LLMs by adding relevant external documents as context, improving the accuracy and richness of outputs.
- **Workflows:** Sequences of interactions between nodes. By designing workflows, users combine multiple nodes into powerful, dynamic AI processes.
- **APIs:** A set of tools to programmatically add nodes, modify workflows or extract data. Offers developers flexibility and seamless integration with other systems.
- **LangSmith:** LangSmith is a dedicated API for managing large language models (LLMs). Provides functions for initializing LLMs, creating conditional logic and optimizing performance.

R.M.K GROUP OF INSTITUTIONS

How LangGraph Scales?

Graph-based architecture: Ensures AI workflows grow without slowing down or losing efficiency.

- **Enhanced decision-making:** Models relationships between nodes, enabling AI agents to learn from past actions and feedback.
- **Increased flexibility:** Open-source design lets developers add new components or adapt existing workflows with ease.
- **Multiagent workflows:** Supports networks of specialized LangChain agents. Tasks can be routed to the right agent, enabling parallel execution and efficient handling of complex, diverse workloads.
- **Decentralized coordination:** This multiagent setup creates a scalable system where automation doesn't rely on a single agent but is distributed across a coordinated network.

State of Agentic AI

- States are the different stages or phases an agentic AI system moves through to achieve a goal autonomously. These steps form a continuous loop, often summarized in a core perceive-reason-act-learn cycle.
- The Stages in the definition and operation of an agentic AI system are:

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

- **Perception (Sense):** The agent gathers information from its environment, which can include user prompts, data from sensors, databases, APIs, or external systems. This step provides the agent with the necessary context and real-time information to understand the current situation.
- **Reasoning:** Using a large language model (LLM) as its "brain," the agent processes the gathered data to understand the context, analyze potential solutions, and formulate a plan. This involves breaking down complex, high-level goals into smaller, manageable sub-tasks.
- **Planning:** Based on its reasoning, the AI develops a strategy or a sequence of steps to achieve its objectives. This is often an iterative process that can adapt if conditions change.
- **Action (Execution):** The AI carries out the planned actions by interacting with external tools, APIs, or physical systems (e.g., sending an email, querying a database, or controlling a robot).
- **Learning and Reflection:** After executing an action, the agent observes the outcome and evaluates whether it was successful. It uses this feedback to improve future decision-making processes and refine its strategies over time. This continuous learning process is often powered by machine learning techniques such as reinforcement learning.

R.M.K GROUP OF INSTITUTIONS

- In a multi-agent system, an additional key "state" or component is Coordination, where multiple specialized agents communicate and share information through a shared memory to work together effectively towards a common objective.

AI Agent Memory

- AI Agent Memory is the ability of an AI agent to store, recall and use information from past interactions to make better decisions in the present and future. Without memory, an agent treats every interaction as if it is the first interaction. With memory, an agent can maintain context, adapt to users and improve over time i.e memory gives AI agent continuity, context-awareness and learning abilities.

Need of Memory by AI Agents

- Many real-world scenarios demand agents to remember and adapt to:
- Keep track of conversations.
- Track progress in multi-step workflows.
- Learn from past feedback and improve.
- To keep the personalization i.e remembering user preferences.

State Schema

- The State Schema refers to the definition of all possible conditions, configurations, or information the AI agent can be in or encounter within its environment. This schema is crucial for the agent's ability to plan, reason, and make autonomous decisions.
- The state schema defines the boundaries of the environment in which the agent operates. By understanding all potential states, the agent can:
- **Predict outcomes:** The agent can better predict the consequences of its actions.
- **Plan effectively:** It can develop a strategy to achieve a specific goal by navigating through different potential states.
- **Adapt to change:** It helps the agent adapt its behavior when the environment changes.

Multiple Schemas:

In agentic AI, particularly in multi-agent systems, employing multiple schemas is common and often necessary. This involves using different schemas for various purposes or for different agents within a system.

Lecture Notes

Purpose:

- To manage complexity, promote modularity, and facilitate specialized interactions. Different agents or different aspects of an agent's operation may require distinct data structures and interaction protocols.
- **Examples:**
- **Tool schemas:** Schemas defining the input and output parameters of tools or APIs that an agent can utilize, allowing the agent to correctly interact with external resources.
- **Communication schemas:** Schemas that standardize the format of messages exchanged between agents in a multi-agent system, ensuring clear and unambiguous communication.
- **Knowledge schemas:** Schemas for representing specific domains of knowledge, allowing agents to store and retrieve information in a structured and efficient manner.
- **Task schemas:** Schemas outlining the steps, dependencies, and expected outcomes of a particular task, guiding an agent's planning and execution.
- **Stateless Reducer**
- In traditional software development (and some AI agents), the system's internal state might be hidden or persist in unpredictable ways. The reducer model proposes a cleaner, more reliable architecture.
- **Pure Function:** The agent acts as a pure function that, given the same inputs, always produces the same outputs without relying on hidden internal state or causing side effects.
- **Explicit State Management:** Instead of maintaining a hidden, long-lived internal memory, the agent takes the *current state* and a new *observation/action* as inputs, and returns a *new state* and observable actions.
- **Formula:** newState = agent(currentState, observation, action).

R.M.K GROUP OF INSTITUTIONS

Trim Messages

Trim Messages is the process of reducing the size of the conversation history by removing older or less relevant messages when the context becomes too long. It keeps only the latest and essential messages so that the model stays within token limits while still understanding the current query.

Features

- Removes older messages to avoid token overflow.
- Keeps only the most recent and relevant conversation context.
- Reduces API cost and improves response speed.
- May replace removed history with a short summary.
- Maintains essential instructions while discarding unnecessary details.

How Does It Work?

The system constantly monitors the total size of the conversation—measured in tokens or message count—before sending it to the LLM. When the size exceeds the allowed limit, the system begins trimming by removing the oldest messages first, because they usually hold less importance for the current query. In more advanced systems, instead of completely deleting old content, a concise summary is generated to preserve essential context. This summary replaces the removed messages, ensuring the model still understands what happened earlier without needing the full original text. This approach maintains conversational flow, reduces memory usage, and keeps the LLM focused on the most relevant parts of the dialogue.

Example

A chatbot has a 20-message conversation. Before sending the 21st user message to the LLM, the system removes the first 10 old messages and keeps only the latest 10. Another example: instead of sending all previous product FAQs, the app replaces them with a short summary like "User asked about refund policies earlier."

Filter Messages

Filter Messages is the process of selectively removing messages that are irrelevant, sensitive, repetitive, or unsafe before sending the conversation to the LLM. This ensures the model receives only useful and appropriate information to generate an accurate response.

Features

- Removes irrelevant or repetitive messages from the history.

- Excludes sensitive or personal information to maintain privacy.
- Filters based on rules such as message role, content type, or relevance.
- Helps improve model accuracy by avoiding noise in the input.
- Ensures safe, clean, and context-focused conversation data.

How Does It Work?

The system evaluates each message using predefined rules or AI-based content checks to decide whether it should be included or removed. During this process, messages containing sensitive information, irrelevant topics, repeated text, or system-generated noise are flagged for exclusion.

The filtering layer then removes these unwanted messages, ensuring they never reach the LLM. Only clean, meaningful, and context-relevant messages are collected into the final conversation history sent to the model. This careful screening improves accuracy, protects privacy, and helps the LLM focus on the information that truly matters for generating the next response.

Example

If a user accidentally shares phone numbers, passwords, or IDs, the system filters out these messages before sending the context to the LLM. Another example: if debug logs like "System: Timeout error at 04:35 PM" appear in the history, the filter removes them because they are irrelevant to the model's task.

Memory and External Memory

Memory in LLM systems refers to the model's ability to store important information from previous interactions or conversations. This "internal memory" is usually saved within the application layer and reused in later responses to maintain continuity. It helps the system remember user preferences, context, and past discussions without needing to re-explain everything.

Features

- Stores small, relevant user details (name, preferences, past answers).
- Helps maintain personalization and smooth conversation flow.
- Limited in size and carefully managed.

How It Works?

Memory works by first identifying useful information from the user's messages, such as preferences, personal details, or important facts shared during the conversation. Once detected, these key details are stored in a dedicated memory module within the system.

- During the next interaction, the system automatically retrieves the relevant stored information and includes it in the prompt it sends to the LLM. As a result, the model uses this recalled data to generate responses that feel more personalized, context-aware, and consistent with previous discussions.

Example : If a user says, "My name is Riya, and I prefer brief answers," the system identifies these as important details and stores them in its memory module. Later, when the user returns with another question, the system retrieves this stored information and includes it in the prompt sent to the LLM. As a result, the AI naturally responds with something like, "Hi Riya! Here's a short answer to your question," showing that it remembers both the user's name and their preference for concise replies. This creates a personalized and more engaging interaction experience.

External Memory : External Memory refers to storage systems outside the LLM—such as databases, vector stores, or knowledge bases—that hold large or long-term information the model cannot fit into its internal context window. This allows the AI to access and use vast amounts of data without embedding everything inside the prompt.

Features

- Supports long-term knowledge retention beyond the LLM's built-in context window.
- Allows storage of large volumes of information such as documents, manuals, logs, and historical records.
- Can handle different data types including structured (tables), semi structured (JSON, XML), and unstructured data (text, PDFs).
- Helps retrieve specific information whenever the user asks, enabling accurate and fact-based responses.
- Provides persistent, scalable storage that does not depend on the LLM's token limits.

How It Works?

When a user sends a query, the system first searches the external memory source—such as a vector database, knowledge base, or document repository— to find the most relevant information related to the question. Once the matching documents or records are retrieved, this information is combined with

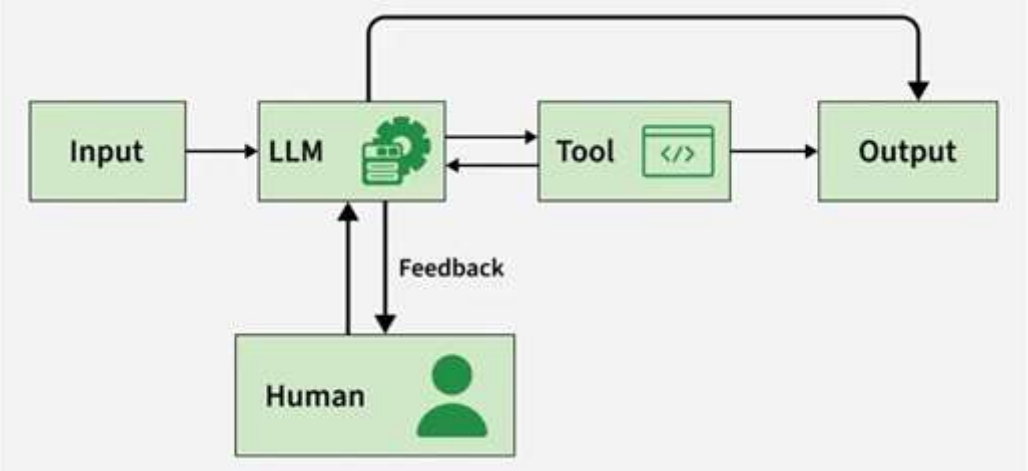
the user's query and passed to the LLM as additional context. With this enriched input, the model can generate a far more accurate and informed answer, using verified external knowledge rather than relying only on its internal training data. This process is commonly known as Retrieval-Augmented Generation (RAG), and it allows AI systems to deliver precise, up-to-date, and domain-specific responses.

Example

For example, imagine a company that stores all its product manuals in a vector database as part of its external memory system. When a user asks a question like, "How do I reset Model X200?", the system immediately searches through this external memory to locate the most relevant section of the manual. After retrieving the exact instructions, it sends that information along with the user's query to the LLM. Using this retrieved content, the AI is able to provide an accurate and detailed reset procedure, even though the full manual itself is not stored inside the model. This approach ensures precise, reliable answers supported directly by the company's own documentation.

Human-in-the-Loop (HITL)

Human-in-the-Loop (HITL) systems combine AI automation with human oversight. Here, humans actively monitor, validate or refine AI outputs during training, testing or decision-making. Even deep learning models can face bias or ambiguity in unseen scenarios. HITL helps tackle these issues by creating a feedback loop where humans continuously evaluate and improve AI decisions. Goal is to merge AI speed and scale with human judgment and ethics for more accurate, transparent and responsible outcomes.



Lecture Notes

How Does HITL Work?

HITL systems bring together AI's predictive power and human insight in a continuous feedback loop. Human involvement whether before, during or after model training helps the system learn better, make smarter decisions and stay reliable.

Workflow

1. Training Phase: Humans label or annotate raw data to teach the model (e.g., "spam" vs. "not spam"). This forms the ground truth dataset.
2. Inference Phase: The trained AI model predicts outcomes for new data.
3. Feedback Phase: Humans review and correct AI outputs, providing feedback that is used to retrain or fine-tune the model forming a continuous improvement loop.

Common HITL workflows:

1. Supervised Learning: Humans step in to label examples like marking emails as "spam" or "not spam" or images as "car" or "bus." These labeled datasets teach the model how to recognize patterns, helping it become more accurate and dependable over time.
2. Reinforcement Learning from Human Feedback (RLHF): Here AI systems learn by combining reinforcement learning with real human judgments. Instead of relying solely on predefined rewards, the model uses human feedback to align its decisions with human values and preferences powering generative AI systems to produce more accurate and human like responses.
3. Active Learning: Here, the AI asks for human help only when it's unsure. By flagging ambiguous cases, it gets precise feedback without wasting effort making the learning process smarter and more efficient.

Human-Guided Training

AI model prediction be

$$\hat{y} = f_\theta(x)$$

where

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

- f_θ denotes the model.
- θ represent the parameters of the model

Human feedback $h(x)$ adjusts or corrects the prediction.

The updated prediction can be represented as:

$$y' = \alpha \cdot \hat{y} + (1 - \alpha) \cdot h(x)$$

where $\alpha \in [0, 1]$ determines the weights

In Reinforcement Learning from Human Feedback (RLHF), the model receives a reward signal rt based on human preference:

$$\theta_{t+1} = \theta_t + \eta \nabla_\theta r_t$$

where η is learning rate.

This allows the AI to align behavior with human-approved outcomes over time.

Why Human-in-the-Loop Matters ?

1. Accuracy and Reliability: Humans play a key role in catching model mistakes and spotting unusual patterns, keeping AI systems reliable even in rare or unpredictable situations. Continuous human feedback helps the model learn from its errors and adapt more effectively over time.

2. Ethical Decision-Making and Accountability: When AI faces morally complex situations like healthcare or justice humans bring ethical reasoning that machines lack. HITL systems also maintain clear audit trails, ensuring transparency and accountability.

3. Transparency and Explainability: By keeping humans involved in validation and decision making HITL makes AI's inner workings easier to understand. When people can see how and why an AI reached a conclusion, it builds trust especially in sensitive fields like finance or medicine.

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Human-in-the-Loop (HITL) Annotation

- Combines automated labeling with human review, ensuring both efficiency and accuracy in creating high-quality datasets.
- Humans identify and fix incorrect or ambiguous labels generated by automated systems, reducing noise in training data.
- Models flag uncertain samples, which are then sent to human annotators for clarification improving learning efficiency.
- Human experts provide context-sensitive labels (e.g., medical, legal or financial data) that automation alone might misinterpret.
- Each round of human correction helps the model learn from its mistakes, enhancing performance over time.
- Human oversight ensures balanced, unbiased annotations, creating datasets that better represent real-world diversity.

Applications

Healthcare Diagnostics: Intelligent systems analyze scans and patient data to suggest possible diagnoses. Doctors then review and confirm results improving speed and accuracy while ensuring patient safety.

Cybersecurity: Smart algorithms monitor network activity for unusual patterns. When a potential threat is detected, human analysts step in to verify and respond, adding precision and contextual understanding.

Fraud Detection: In finance, predictive models flag suspicious transactions. Human experts review these alerts to prevent false positives and maintain fairness in decision-making.

Customer Support: Virtual assistants handle routine questions, while human agents take over complex or emotionally sensitive issues delivering both quick responses and genuine empathy.

Challenges

- **Scalability and Cost:** Human annotation and oversight can be slow and expensive, especially with massive datasets or specialized domains like radiology or law.
- **Human Error and Inconsistency:** Humans introduce their own biases, fatigue and subjectivity, which may reduce labeling consistency.
- **Privacy and Security Risks:** Exposing sensitive data to human reviewers may increase the potential for data leaks or misuse.

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Building Agent with LangGraph

LangGraph is a companion framework designed to build complex workflows for AI applications in a visual and modular way. It lets developers:

- Create graph-based state machines for LLM workflows.
- Define nodes for each logical step (e.g., classify, generate, refine).
- Handle stateful data flow across the system.
- Visualize the entire workflow as a graph.

Together, LangChain and LangGraph create a modular setup for building powerful, maintainable AI systems.

Step 1: Install the dependencies

We will install the required packages such as LangChain, LangGraph and matplotlib that is required for the implementation.

[No Title]

```
!pip install langchain langgraph langchain-openai networkx matplotlib
```

Step 2: Setup OpenAI Key

We need to setup our OpenAI key so that langchain can authenticate requests.

```
import os
os.environ["OPENAI_API_KEY"] = "YOUR_OPENAI_API_KEY"
```

Step 3: Define the Agent State

We will define the agent state:

- question: stores the user's latest input.
- answer: holds the model's generated response.
- history: keeps all previous conversation turns for contextual replies.

```
from typing_extensions import TypedDict, List
class State(TypedDict):
    question: str
    answer: str
    history: List[str]
```

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Step 4: Initialize the LLM

We will:

• Uses GPT-4.1 model from OpenAI.

• temperature=0.3 keeps responses focused and consistent (low randomness).

```
from langchain.chat_models import init_chat_model
llm = init_chat_model("openai:gpt-4.1", temperature=0.3)
```

Step 5: Define Node Functions for the Workflow

Here:

• classify(): placeholder step that could later handle intent detection or routing.

• generate(): builds the full prompt using conversation history + current question, invokes the model and returns an answer.

• refine(): slightly improves clarity and appends the interaction to memory (history).

```
def classify(state: State):
    return {"question": state["question"]}

def generate(state: State):
    history_context = "\n".join(state.get("history", []))
    prompt = f"""

You are a conversational AI assistant. Use conversation history to reply naturally.
History:
{history_context}
Question:
{state['question']}
Answer:"""
        response = llm.invoke([{"role": "user", "content": prompt}])
        return {"answer": response.content}
def refine(state: State):
    refined = state["answer"] + "\n\n[Refined for clarity]"
    history = state.get("history", [])
    history.append(f"Q: {state['question']}\nA: {refined}")
    return {"answer": refined, "history": history}
```

R.M.K GROUP OF INSTITUTIONS

Lecture Notes

Step 6: Build the LangGraph Workflow

We will define:

- StateGraph(State): creates a graph that handles state of type State.
- add_sequence(): connects nodes in order.
- add_edge(): sets the graph's start point.
- compile(): finalizes it for execution.

```
from langgraph.graph import START, StateGraph
graph_builder = StateGraph(State).add_sequence([classify, generate, refine])
graph_builder.add_edge(START, "classify")
graph = graph_builder.compile()
```

Step 7: Visualize the Workflow

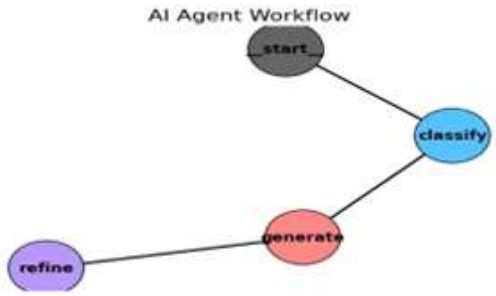
We will visualize the workflow:

- NetworkX is used to represent the nodes (classify, generate, refine).
- Each node is displayed with color and arrows to show data flow.
- The output visual helps understand the internal logic of the AI Agent.

```
import networkx as nx
import matplotlib.pyplot as plt
def visualize_langgraph(graph_builder):
    G = nx.DiGraph()
for node in graph_builder.nodes:
    G.add_node(node)
for src, tgt in graph_builder.edges:
    G.add_edge(src, tgt)
pos = nx.spring_layout(G, seed=42, k=1.2)
colors = ["#56c2ff", "#ff8888", "#b996fa"]
sizes = [3000] * len(G.nodes)
nx.draw_networkx_nodes(G, pos, node_color=colors[:len(
    G.nodes)], node_size=sizes, edgecolors='black')
nx.draw_networkx_edges(G, pos, arrows=True, width=2)
```

```
        nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')
        plt.title("AI Agent Workflow", fontsize=16)
        plt.axis('off')
        plt.show()
visualize_langgraph(graph_builder)
```

OUTPUT



Step 8: Run the Stateful AI Agent

- Maintains a state variable across turns (memory).
- Passes input to the LangGraph workflow.
- Updates memory (state.update(response)) after every interaction.
- Prints answers and continues conversation seamlessly.

Memory Schemas

In LangGraph, the "memory schema" primarily refers to the AgentState (often a TypedDict or @dataclass) that defines the structure of data passed between nodes in the graph. This state is managed through a checkpointer, which persists the state across interactions, providing short-term memory (within a single thread) and enabling long-term memory integrations.

Long-Term Memory (Enduring Storage)

Agentic AI — systems designed to act autonomously, make decisions, and pursue goals — relies on various types of memory to function effectively. Drawing from cognitive science concepts, these memory types include

- semantic
- episodic
- short-term
- procedural and
- long-term memory.

Each plays a distinct role in enabling an AI agent to process information, learn, and adapt. Below, I'll explain how these memory types apply to agentic AI in a straightforward way.

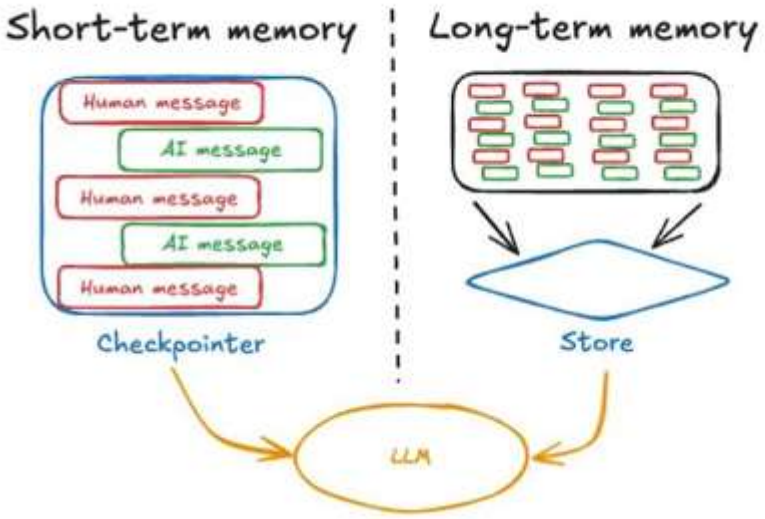
Long-term memory serves as the AI's repository for knowledge accumulated over time. Unlike short-term memory, it's designed to persist, storing information that the agent can revisit later. In agentic AI, long-term memory might include learned patterns, past experiences, or external data it has been trained on. For instance, an AI that remembers how it solved a complex query last week can apply that knowledge to a similar problem today. It's the foundation for growth and adaptability, allowing the agent to build expertise.

Short-TermMemory

Short-term memory in agentic AI is like a temporary holding area for information needed right now. It's where the AI keeps track of immediate inputs, such as the current state of a task or the latest user command.

For example, if you ask an AI to solve a problem step-by-step, short-term memory stores the details of the ongoing process. It's limited in capacity and duration, typically resetting after the task is complete or the context shifts. This aligns with the "working memory" concept, enabling quick, real-time responses.

Lecture Notes



Long-Term Memory vs Short-Term Memory

Feature	Short-Term Memory	Long-Term Memory
Scope	Single conversational thread/session	Across multiple threads/sessions
Persistence	Persists within a thread via checkpointers	Persists across sessions using external memory stores
Content	Conversation history, current stateful data	User profiles, preferences, domain-specific knowledge
Purpose	Maintain immediate conversational context	Personalize interactions, build persistent knowledge

Implementation	Managed within the agent's state and checkpointers	Custom namespaces and memory stores

Memory Deployment Considerations

Persistence:

Choose a memory store that aligns with the persistence requirements of the application. InMemoryStore is suitable for non-critical data in development, while vector databases or external databases like Redis are necessary for persistent, scalable solutions in production.

Scalability:

For large-scale applications, consider the scalability of the chosen memory store. Vector databases are designed for efficient knowledge retrieval and semantic search across extensive datasets.

Resource Management:

Monitor and manage resource utilization, especially when using external databases like Redis. Configure concurrent operations and monitor metrics to prevent memory issues and optimize performance.

Compatibility:

Ensure compatibility between LangGraph and the chosen memory store and any associated dependencies. For example, using Redis instead of Valkey with AWS ElastiCache is recommended to avoid potential issues.

Deployment Options:

LangGraph applications, including their memory components, can be deployed through options like LangSmith Deployment (Cloud, Hybrid, or Fully Self-Hosted) or on custom infrastructure.

Self-Learning / Enrichment Topics

1. Agentic AI as Active Intelligence Systems

Beyond textbook definitions, Agentic AI represents the transition from *thinking machines* to *acting systems*.

Key idea:

Intelligence is no longer measured by output quality alone, but by **goal achievement over time in open environments**.

Advanced perspective:

- Agents operate under **uncertainty**
- Goals may evolve dynamically
- Success depends on *long-horizon planning* and *self-correction*

2. Autonomous Goal Formation & Self-Motivation

Most syllabus content assumes goals are given. Advanced agentic systems explore:

- **Goal discovery** from environmental signals
- **Intrinsic motivation** (curiosity-driven agents)
- **Utility shaping** based on experience

Examples:

- Research agents identifying promising hypotheses
- Trading agents redefining risk profiles autonomously

Learning agents prioritizing exploration vs exploitation

R.M.K GROUP OF INSTITUTIONS

3. Hierarchical & Meta-Agent Architectures

Beyond single agents:

Hierarchical Agents

- High-level planner → mid-level task agents → low-level execution agents
- Similar to organizational management structures

Meta-Agents

- Agents that **design, evaluate, or improve other agents**
- Perform self-debugging and performance optimization

Use cases:

- Auto-AI pipelines
- Self-improving code agents
- Autonomous research labs

4. Long-Term Memory, Episodic Recall & Identity

Beyond short-context memory:

Advanced memory types:

- **Episodic memory** (past experiences)
- **Semantic memory** (abstract knowledge)
- **Procedural memory** (skills & workflows)
- **Identity memory** (preferences, norms, behavioral consistency)

Why it matters:

- Enables personality continuity
- Supports long-running agents (weeks/months)
- Prevents repetitive mistakes

R.M.K GROUP OF INSTITUTIONS

Mini Project / Activity

1. Autonomous Research Assistant Agent

Objective

Design an agent that autonomously performs literature review on a given topic.

Agentic Features

Goal decomposition (search → filter → summarize)
Tool usage (search, PDF reader)
Reflection loop (improve summary quality)
Memory of previously reviewed papers

Implementation Stack

LLM (OpenAI / Llama)
LangChain / CrewAI / AutoGen
Vector DB (FAISS / Chroma)

Deliverables

Topic-wise research summary
Agent workflow diagram
Reflection log

R.M.K GROUP OF INSTITUTIONS

PART A- UNIT-3

1. What is a Prompt Template in LangChain?

A prompt in LangChain is the structured input given to an LLM to guide its response. It can include instructions, context, examples, or variables that dynamically fill user data. Good prompts increase accuracy and reduce hallucinations. LangChain provides PromptTemplates for reusable prompts. They help maintain consistency across chains and agents.

2. What are Chains in LangChain?

Chains are sequences of steps where the output of one component becomes the input of the next. They help organize complex LLM workflows into modular units. A chain can include prompts, models, tools, and post-processing. LangChain offers SimpleSequentialChain and LLMChain types. Chains improve reusability and clarity.

3. What are Tools in LangChain?

Tools are external functions that an agent can call to perform specific actions like search, calculator, APIs, or DB queries. They extend the capability of LLMs beyond text generation. Tools are wrapped so the agent can call them through structured responses. They help solve real-world tasks. Tools are central to agent workflows.

4. What is an Agent in LangChain?

An agent is an LLM-powered decision-maker that chooses which tool to use based on user input. It plans steps dynamically instead of following a fixed chain. Agents use prompts, tools, and memory to reason. They follow ReAct or tool-calling patterns. Agents help solve open-ended tasks like research or planning.

5. What is Memory in LangChain?

Memory stores past conversation or context so the model can maintain continuity. It helps create personalized, context-aware interactions. Types include BufferMemory, SummaryMemory, and EntityMemory. Memory reduces repetitive prompts and improves long conversations. It can also extend to vector stores for long-term recall.

6. What are Runnables in LangChain?

Runnables provide a flexible, composable way to build pipelines using the LCEL framework. Each component becomes a runnable block that can be chained, mapped, or branched. They improve modularity and reusability. Runnables support parallel execution and streaming. They simplify building complex workflows without custom classes.

7.What is LCEL (LangChain Expression Language)?

LCEL is a declarative way to connect prompts, models, tools, and logic into a pipeline. It allows simple symbols like | to chain components. LCEL supports composition, branching, fallback, and input/output mapping. It executes efficiently with batch and streaming support. LCEL makes pipelines faster and more readable.

8. What is LangGraph?

LangGraph is a framework for building stateful agents using graph-based workflows. Each node represents a step, and edges define the flow based on conditions. It supports memory, reducers, and external data stores. LangGraph is ideal for multi-step reasoning and agent coordination. It ensures predictable state transitions.

9. What is State Schema and State Reducer in LangGraph?

State Schema defines the structure of data stored between nodes, like messages or variables. State Reducer updates this state after each node runs. It merges old and new information using functions like append or overwrite. This enables controlled memory updates. Together, they manage agent state efficiently.

10. What is Human-in-the-Loop (HITL) in AI Workflows?

HITL means involving a human in reviewing, correcting, or approving LLM outputs. It improves safety, accuracy, and trustworthiness in critical workflows. LangGraph supports HITL by adding review nodes. Humans can override decisions or validate tool calls. It ensures high-quality responses in sensitive applications.

R.M.K GROUP OF INSTITUTIONS

PART-B QUESTIONS

1. Explain the **core architecture of LangChain** in detail. Discuss the role of **Prompts, Chains, Tools, Agents, Memory, and Runnables**, and explain how **LCEL (LangChain Expression Language)** helps in composing complex workflows. Illustrate your answer with a suitable example.

2. Explain the concept of **Memory in LangChain**.
   Differentiate between: Short-term memory Long-term memory
   Also explain how memory is used while building a **chatbot with memory**, highlighting its importance in maintaining conversation context.

3. Explain how **LangChain integrates with different LLM service providers** such as: OpenAI, Google Gemini, Ollama, Hugging Face, Claude. Discuss the common integration pattern, challenges, and benefits of provider-agnostic design.

4. Explain the mechanisms of **Trim and Filter Messages** in LangGraph.
   Why are they important in long-running agent systems?
   Also explain the use of **External Memory** and its role in scaling agent-based applications.

5. Explain **State and Memory management in LangGraph**.
   Discuss the following in detail: State Schema Multiple Schemas State Reducer
   Explain how state reducers help in managing updates across multiple nodes in a workflow.

R.M.K GROUP OF INSTITUTIONS

Assignment-I (Toppers)

Q. No.	Question	CO Level	K Level
1	Analyze the complete LangChain execution flow using Runnables and LCEL. Explain how LCEL improves modularity, composability, and scalability compared to traditional chain-based designs.		
2	Critically evaluate memory handling in LangChain vs LangGraph. Explain how external memory, message trimming, and filtering help in building scalable long-running agents.		
3	Design a complete agent using LangGraph that includes: Multiple states and schemas Short-term and long-term memory Human-in-the-Loop interaction Deployment considerations Explain the design decisions and workflow clearly.		

Assignment-I (Above Average)

Q. No.	Question	CO Level	K Level
1	Explain State and Memory management in LangGraph. Discuss the role of State Schema and State Reducers with a simple workflow.		
2	Explain how Agents and Tools interact in LangChain to perform tasks dynamically. Illustrate with a use case where an agent converts natural language into SQL.		
3	Explain how Human-in-the-Loop (HITL) is implemented in LangGraph. Discuss its importance in agent reliability and decision-making.		

GROUP OF INSTITUTIONS

Assignment-I (Average)

Q. No.	Question	CO Level	K Level
1	Describe the LangChain architecture. Explain the roles of Prompts, Chains, Tools, Agents, and Memory in building an LLM-based application.		
2	Explain how memory is used in building a chatbot with LangChain. Describe the flow of information between user input, memory, and response generation.		
3	Describe the process of creating a simple LangGraph workflow. Explain how state flows between nodes.		

R.M.K GROUP OF INSTITUTIONS

Assignment-I (Below Average)

Q. No.	Question	CO Level	K Level
1	Differentiate between short-term memory and long-term memory in LangChain with suitable examples.		
2	Explain how Prompt Templates and Chains work together in LangChain. Illustrate your answer with a simple text-generation example.		
3	Explain the basic components of LangGraph: Nodes Edges State		

GROUP OF INSTITUTIONS

Assignment-I (Slow Learner)

Q. No.	Question	CO Level	K Level
1	Explain the following terms in LangChain with simple definitions and one example each: Prompt Chain Tool Memory		
2	What is memory in LangChain? Explain why memory is needed in chat-based applications.		
3	What is LangGraph? List any four features of LangGraph.		

R.M.K GROUP OF INSTITUTIONS

ASSIGNMENT
UNIT III

1. Design a prompt engineering strategy for improving accuracy in an LLM-based QA system.(K6)
2. Analyze how chaining multiple tools in LangChain affects latency and overall reliability of an agent pipeline. (K4)
3. Evaluate different memory types in LangChain(BufferMemory, SummaryMemory, EntityMemory) and determine which is optimal for long conversations. (K5)
4. Design a LangChain Agent that uses search tools + calculator tools + document retrieval to answer user queries. Draw the architecture. (K6)
5. Analyze the role of Runnables in LangChain and explain how they make pipelines more reusable and composable. (K4)

R.M.K GROUP OF INSTITUTIONS

Supportive online Certification courses

S No	Course provider	Course title	Link
1	Udemy	LangChain Crash Course: Build OpenAI LLM powered Apps	LangChain Crash Course: Build OpenAI LLM powered Apps
2	Hugging Face	**Introduction to LangGraph**	Introduction to LangGraph - Hugging Face Agents Course
3	Udemy	LangChain- Develop AI Agents with LangChain & LangGraph	LangChain- Develop AI Agents with LangChain & LangGraph
4	Coursera	**Building Autonomous AI Agents with LangGraph**	Building Autonomous AI Agents with LangGraph | Coursera
5	edX	**Mastering Generative AI: Agents with RAG and LangChain**	IBM: Mastering Generative AI: Agents with RAG and LangChain | edX

R.M.K GROUP OF INSTITUTIONS

Prescribed Text Books & Reference Books

TEXT BOOKS:

1. The Agentic AI Bible: The Complete and Up-to-Date Guide to Design, Build, and Scale Goal-Driven, LLM-Powered Agents that Think, Execute and Evolve by Thomas R. Caldwell
2. Agentic Artificial Intelligence: Harnessing AI Agents to Reinvent Business, Work and Life by Pascal Bornet, Jochen Wirtz , Thomas H. Davenport , David De Cremer
3. Building Agentic AI Systems: Create intelligent, autonomous AI agents that can reason, plan, and adapt by Anjanava Biswas ,Wrick Talukdar
4. AI Agents in Action by Micheal Lanham
5. The Age of Agentic AI: A Practical & Exciting Exploration of AI Agents by One Book Ahead Publishing

REFERENCE BOOKS:

a. What is Agentic AI – GeeksforGeeks

b. The Agentic AI Handbook: A Beginner's Guide to Autonomous Intelligent Agents

c. What is Agentic AI? | IBM

d. Large Language Model (LLM) Tutorial – GeeksforGeeks

R.M.K GROUP OF INSTITUTIONS

Assessment Schedule

- **Tentative schedule for the Assessment During 2024-2025 odd semester**

S.NO	Name of the Assessment	Start Date	End Date	Portion
1	IAT 1			UNIT 1 & 2
2	IAT 2			UNIT 3 & 4
3	MODEL			ALL 5 UNITS

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

R.M.K GROUP OF INSTITUTIONS